

Instituto Politécnico Nacional
Escuela Superior de Cómputo

Programa Universo

Alumno: [REDACTED] Raul Enrique

No. de Boleta: [REDACTED]

Grupo: [REDACTED]

Unidad de Aprendizaje: Teoría de la Computación

Profesor: Genaro [REDACTED]



Ciudad de México, 23 de octubre de 2025

Índice

1. Introducción	2
2. Marco Teórico	2
3. Desarrollo	2
3.1. Programa Universo	2
3.2. Planteamiento	3
3.3. Implementación en C	3
3.4. Implementación en Python	8
3.5. Algoritmo utilizado	12
4. Resultados	13
5. Conclusión	13
6. Referencias Bibliográficas	14

1. Introducción

En el estudio de los lenguajes formales dentro de la teoría de la computación, se exploran las bases que permiten a las máquinas interpretar y procesar información. Un lenguaje formal se define como un conjunto de cadenas construidas a partir de un alfabeto específico, en este caso el alfabeto binario $\Sigma = \{0, 1\}$. Este trabajo presenta un programa que genera estadísticas analíticas de las cadenas binarias de longitud n (Σ^n) y analiza la densidad de unos por posición, graficando los resultados para $n = 0$ a $n = 33$.

El programa se divide en dos componentes principales:

- Un módulo en C que genera todas las combinaciones binarias para un n dado, guardándolas en un archivo de texto en notación de conjunto.
- Un módulo en Python que calcula analíticamente las densidades de unos por posición en las cadenas binarias, sin necesidad de leer el archivo completo, y produce gráficos de densidades, logaritmo natural, medias y varianzas por n .

El objetivo es comprender cómo los lenguajes formales pueden representarse y analizarse computacionalmente, aplicando conceptos como la generación de Σ^n y el análisis estadístico de cadenas.

2. Marco Teórico

- **Lenguaje (Σ):** Conjunto de símbolos que conforman un alfabeto. En este caso, $\Sigma = \{0, 1\}$.
- **Potencia de un alfabeto (Σ^k):** Conjunto de todas las cadenas de longitud k formadas por símbolos de Σ . Por ejemplo:
 - $\Sigma^0 = \{\epsilon\}$ (cadena vacía).
 - $\Sigma^1 = \{0, 1\}$.
 - $\Sigma^2 = \{00, 01, 10, 11\}$.
 - $\Sigma^3 = \{000, 001, 010, 011, 100, 101, 110, 111\}$.
- Σ^* : Unión de todas las potencias de Σ , es decir, $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$.
- **Cadena:** Secuencia finita de símbolos de Σ .
- **Cadena vacía (ϵ):** Cadena sin símbolos.
- **Longitud de una cadena ($|s|$):** Número de símbolos en la cadena.
- **Lenguaje (L):** Subconjunto de Σ^* , es decir, un conjunto de cadenas.

3. Desarrollo

3.1. Programa Universo

El programa genera estadísticas del universo de cadenas binarias Σ^n para un n dado, ya sea manualmente (el usuario ingresa n) o automáticamente (genera cadenas para $n = 0$ hasta un límite). Los requisitos son:

1. Modos automático y manual.
2. Preguntar al usuario si desea calcular otra n o salir.
3. Guardar las cadenas en un archivo de texto en notación de conjunto ($\{\}$).
4. Para $n = 33$, graficar la densidad de unos por posición, el logaritmo natural de la densidad, y las medias y varianzas por n .
5. Incluir el código fuente en el reporte.

3.2. Planteamiento

El problema se resuelve en dos partes:

- **Generación de cadenas (C):** Se genera Σ^n usando operaciones de desplazamiento de bits para producir todas las combinaciones binarias de longitud n . Las cadenas se guardan en un archivo de texto.
- **Análisis y graficación (Python):** Se calculan analíticamente las densidades de unos por posición para $n = 0$ a $n = 33$, generando gráficos de densidades, logaritmo natural, medias y varianzas.

La implementación en C es eficiente para generar grandes cantidades de cadenas, mientras que Python calcula las estadísticas analíticamente, evitando la lectura de archivos grandes, y usa bibliotecas como Matplotlib y NumPy para la visualización.

3.3. Implementación en C

El programa en C (`creacionDeCadenas.c`) genera todas las combinaciones binarias de longitud n y las guarda en un archivo de texto. Ofrece dos modos:

- **Manual:** El usuario ingresa un n específico (hasta 34 para evitar archivos excesivamente grandes).
- **Automático:** Genera cadenas para $n = 0$ hasta un n límite ingresado.

A continuación, el código fuente completo:

```

1 //RAUL ENRIQUE MARTINEZ CRUZ 4CM2
2 //PROGRAMA DE COMBINACIONES BINARIAS
3
4 #include<stdio.h>
5 #include<stdlib.h>
6 #include<string.h>
7 #include<unistd.h> //para sleep
8
9 //-----Prototipos de funcion-----
10 void generarCadenas(int n, const char* filename);
11 void generarCadenasAuto(int n);
12
13 //-----F U N C I O N E S-----
14

```

```

15 // Genera combinaciones binarias de un n especifico (consola +
    archivo)
16 void generarCadenas(int n, const char* filename){
17     long long total = 1LL<<n; //<<N nos servira para recorrer a la
        izquierda n posiciones y asi definir 2^n
18                                     //sin necesidad de hacer calculos y
        agregar mas procesos al codigo o
        cargar la memoria
19                                     //tambien tendremos como beneficio que
        todos estos calculos no se
        guardaran en memoria
20                                     //por lo cual podremos manejar N's muy
        grandes sin comprometer nuestro
        equipo

21
22     FILE *f = fopen(filename, "w");
23     if (!f) {
24         printf("Error al abrir el archivo %s.\n", filename);
25         return;
26     }
27
28     // Memoria dinamica para la cadena
29     char *cadena = (char*) malloc((n+1) * sizeof(char));
30     if (!cadena) {
31         printf("Error de memoria.\n");
32         fclose(f);
33         return;
34     }
35     cadena[n] = '\0';
36
37     fprintf(f, "\nn=%d\n\n", n);
38     //printf("\nn=%d\n\n", n);
39
40     // Abrimos el conjunto
41     fprintf(f, "{");
42     // printf("{");
43
44     for (long long i = 0; i < total; i++) {
45         for (int j = n-1; j >= 0; j--) {
46             cadena[n-1-j] = ((i >> j) & 1) ? '1' : '0';
47         }
48         if (i < total - 1) {
49             fprintf(f, "%s, ", cadena);
50             // printf("%s, ", cadena);
51         } else {
52             fprintf(f, "%s", cadena);
53             //printf("%s", cadena);
54         }
55     }
56
57     // Cerramos el conjunto

```

```

58     fprintf(f, "}\n");
59     printf("}\n");
60
61     free(cadena);
62     printf("\n\nCadenas generadas correctamente, gracias por usar
        nuestro sistema:))\n");
63     fclose(f);
64     printf("\n\nCadenas guardadas en %s\n\n\n", filename);
65     for(int k=0;k<8;k++){
66         printf(".");
67         sleep(1);
68     }
69 }
70
71 // Genera combinaciones desde n=0 hasta el n dado (solo archivo)
72 void generarCadenasAuto(int n){
73     FILE *f = fopen("combinaciones.txt", "w");
74     if (!f) {
75         printf("Error al abrir el archivo.\n");
76         return;
77     }
78
79     for (int k = 0; k <= n; k++) {
80         long long total = 1LL << k;
81         char *cadena = (char*) malloc((k+1) * sizeof(char));
82         if (!cadena) {
83             printf("Error de memoria.\n");
84             fclose(f);
85             return;
86         }
87         cadena[k] = '\0';
88
89         fprintf(f, "\n\nN=%d\n\n", k);
90
91         // Abrimos conjunto
92         fprintf(f, "{");
93
94         for (long long i = 0; i < total; i++) {
95             for (int j = k-1; j >= 0; j--) {
96                 cadena[k-1-j] = ((i >> j) & 1) ? '1' : '0';
97             }
98             if (i < total - 1) {
99                 fprintf(f, "%s, ", cadena);
100             } else {
101                 fprintf(f, "%s", cadena);
102             }
103         }
104
105         // Cerramos conjunto
106         fprintf(f, "}\n");
107

```

```

108     free(cadena);
109 }
110 printf("\nCadenas generadas correctamente, gracias por usar
    nuestro sistema:)))");
111 fclose(f);
112 printf("\nCadenas guardadas en combinaciones.txt\n\n");
113 for(int k=0;k<5;k++){
114     printf(".");
115     sleep(1);
116 }
117 }
118
119 //-----M A I N-----
120 int main(){
121     int opc;
122
123     do {
124         system("cls");
125         printf("\n
            =====
126         printf("\nBienvenido a su programa de generacion de cadenas
            binarias");
127         printf("\n
            =====\n");
128         sleep(1);
129         printf("\n-----Menu-----");
130         printf("\n1) Arranque automatico dado 'N' limite");
131         printf("\n2) Manual dado 'N' especifico");
132         printf("\n3) Salir\n");
133         printf("Opcion: ");
134         scanf("%d",&opc);
135
136         switch (opc) {
137         case 1: { // automatico
138             int max;
139             system("cls");
140             printf("\nIngresa un 'N' limite: ");
141             scanf("%d",&max);
142
143             if(max < 0 || max > 34){ // limite razonable para
                evitar archivos gigantes
144                 printf("Ingrese un numero valido (0-34).\n");
145                 for(int k=0;k<3;k++){
146                     printf(".");
147                     sleep(1);
148                 }
149             }
150             else {
151                 generarCadenasAuto(max);

```

```

152     }
153     break;
154 }
155
156 case 2: { // manual
157     int n;
158     system("cls");
159     printf("Ingrese un numero de n combinaciones que desea
160           ver: ");
161     scanf("%d", &n);
162
163     if(n < 0 || n > 34){
164         printf("Ingrese un numero valido (0-34).\n");
165         for(int k=0;k<3;k++){
166             printf(".");
167             sleep(1);
168         }
169     }
170     else{
171         const char* filename = "combinaciones.txt";
172         generarCadenas(n, filename);
173     }
174     break;
175 }
176
177 case 3:
178     printf("\nSaliendo");
179     for(int k=0;k<3;k++){
180         printf(".");
181         sleep(1);
182     }
183     return 0;
184
185 default:
186     printf("Opcion no valida.\n");
187 } while (1);
188
189 return 0;
190 }

```

Listing 1: Código en C para generar cadenas binarias

Explicación del código en C:

- **Líneas 1-6:** Inclusión de bibliotecas estándar (stdio.h, stdlib.h, string.h) y unistd.h para la función sleep.
- **Líneas 8-9:** Prototipos de las funciones generarCadenas (modo manual) y generarCadenasAut (modo automático).
- **Líneas 12-45:** La función generarCadenas genera 2^n cadenas binarias usando desplazamiento de bits (y &). Usa memoria dinámica para almacenar cada cadena y escribe el resultado

en un archivo en formato $\{cadena_1, cadena_2, \dots\}$.

Líneas 48-80: La función `generarCadenasAuto` genera cadenas para $n = 0$ hasta un límite dado, escribiéndolas en `combinaciones.txt`.

Líneas 83-128: El `main` presenta un menú interactivo para elegir entre modo manual, automático o salir. Valida que $n \leq 34$ para evitar archivos excesivamente grandes.

Algoritmo clave: Usa desplazamiento de bits (`1LL<<n`) para calcular 2^n y generar cadenas eficientemente sin cálculos intermedios pesados.

3.4. Implementación en Python

El programa en Python (`graficador.py`) calcula analíticamente las densidades de unos por posición en las combinaciones binarias para $n = 0$ a $n = 33$, sin necesidad de leer un archivo de entrada, ya que este sería demasiado grande (por ejemplo, 32 GB para $n = 33$). Genera cuatro gráficos:

1. Mapa de calor de la densidad de unos por posición.
2. Mapa de calor del logaritmo natural de la densidad.
3. Media de las densidades por n .
4. Varianza de las densidades por n .

A continuación, el código fuente:

```
1 import numpy as np                #para arreglos numericos y
   calculos matematicos
2 import matplotlib.pyplot as plt   #para generar graficos
3 import time                       #para medir tiempos de ejecucion
4 import os                         #para verificar si existe un
   archivo de entrada
5
6 # Nombre del archivo de texto a leer
7 INPUT_FILE = "binary_stats.txt"
8
9 def read_binary_stats_from_file(filename, max_n):
10     """
11     Lee estadísticas de combinaciones binarias desde un archivo txt
12     .
13     Formato esperado: una línea por n, con n valores separados por
14     comas (suma de 1s).
15     Devuelve una lista de arreglos stats y totales.
16     """
17     # Dado que el archivo es demasiado grande, ignoramos la lectura
18     y siempre generamos.
19     print("El archivo es demasiado grande para leer en memoria.
20           Usando calculo analítico en su lugar.")
21     return None
22
23 def generate_binary_stats(n, sample_size=None):
```

```

20 """
21 Calcula estadísticas exactas de combinaciones binarias sin
padding (unpadded) para n meros de 1 a 2^n - 1.
22 Devuelve arreglos de stats (suma de 1s por posición), counts (
n mero de cadenas con esa posición) y total.
23 Posiciones alineadas a la izquierda (MSB en pos=0).
24 """
25 if n == 0:
26     return np.array([0]), np.array([1]), 1 # Caso especial para
n=0
27
28 stats = np.zeros(n, dtype=np.int64)
29 counts = np.zeros(n, dtype=np.int64)
30 total = (1 << n) - 1 # 2**n - 1
31
32 for pos in range(n):
33     if pos == 0:
34         counts[pos] = total
35         stats[pos] = total
36     else:
37         counts[pos] = (1 << pos) * ((1 << (n - pos)) - 1)
38         stats[pos] = (1 << (pos - 1)) * ((1 << (n - pos)) - 1)
39
40 return stats, counts, total
41
42 def plot_binary_combinations(n_range, sample_size=250000):
43     """
44     Genera cuatro gráficas:
45     1. Mapa de calor de la densidad de 1s.
46     2. Mapa de calor del logaritmo natural de la densidad.
47     3. Media de las densidades por n.
48     4. Varianza de las densidades por n.
49     """
50     max_n = max(n_range)
51     heatmap_data = np.zeros((len(n_range), max_n)) # Densidades
52     log_heatmap_data = np.zeros((len(n_range), max_n)) # Logaritmo
de densidades
53     means = np.zeros(len(n_range)) # Media por n
54     variances = np.zeros(len(n_range)) # Varianza por n
55
56     for i, n in enumerate(n_range):
57         start_time = time.perf_counter() # Cambiado a perf_counter
para mayor precisión
58         stats, counts, total = generate_binary_stats(n, sample_size)
59
60         # Calcular densidad por posición: stats / counts (ya que
counts var a por posición en unpadded)
61         density = np.zeros(n)
62         for j in range(n):
63             density[j] = stats[j] / counts[j] if counts[j] > 0 else
0

```

```

64
65     # Ajustar tama o para que coincida con max_n
66     padded_density = np.zeros(max_n)
67     if n > 0:
68         padded_density[:n] = density[:n]
69     else:
70         padded_density[0] = 0
71     heatmap_data[i, :max_n] = padded_density
72     log_density = np.log(padded_density + 1e-10) # Evitar ln(0)
73     log_heatmap_data[i, :max_n] = log_density
74     means[i] = np.mean(density) if n > 0 else 0
75     variances[i] = np.var(density) if n > 0 else 0
76     print(f"n={n}, Tiempo: {time.perf_counter() - start_time:.4f
77           } segundos") # Cambiado a perf_counter y formato .4f
78
79 # Gr fica 1: Mapa de calor de densidad
80 plt.figure(figsize=(12, 8))
81 plt.imshow(heatmap_data, cmap='viridis', aspect='auto')
82 plt.colorbar(label='Densidad de 1s')
83 plt.xlabel('Posici n en la secuencia (0=MSB izquierda)')
84 plt.ylabel('Valor de n')
85 plt.title('Densidad de 1s en combinaciones binarias sin padding
86           (n=0 a 33)')
87 plt.xticks(np.arange(max_n), np.arange(max_n))
88 plt.yticks(np.arange(len(n_range)), n_range)
89 plt.show()
90
91 # Gr fica 2: Mapa de calor de logaritmo natural
92 plt.figure(figsize=(12, 8))
93 plt.imshow(log_heatmap_data, cmap='plasma', aspect='auto')
94 plt.colorbar(label='Logaritmo natural de la densidad')
95 plt.xlabel('Posici n en la secuencia (0=MSB izquierda)')
96 plt.ylabel('Valor de n')
97 plt.title('Logaritmo natural de la densidad de 1s (n=0 a 33)')
98 plt.xticks(np.arange(max_n), np.arange(max_n))
99 plt.yticks(np.arange(len(n_range)), n_range)
100 plt.show()
101
102 # Gr fica 3: Media de densidades
103 plt.figure(figsize=(10, 6))
104 plt.plot(n_range, means, marker='o', linestyle='--', color='b')
105 plt.xlabel('Valor de n')
106 plt.ylabel('Media de densidades')
107 plt.title('Media de densidades de 1s por n (0 a 33)')
108 plt.grid(True)
109 plt.xticks(n_range)
110 plt.show()
111
112 # Gr fica 4: Varianza de densidades
113 plt.figure(figsize=(10, 6))
114 plt.plot(n_range, variances, marker='o', linestyle='--', color='r'

```

```

    ')
113 plt.xlabel('Valor de n')
114 plt.ylabel('Varianza de densidades')
115 plt.title('Varianza de densidades de 1s por n (0 a 33)')
116 plt.grid(True)
117 plt.xticks(n_range)
118 plt.show()
119
120 def main():
121     n_range = range(34) # 0 a 33
122     sample_size = 250000 # No se usa, ya que calculamos
                           analiticamente
123
124     print("Generando graficas de combinaciones binarias usando
           calculo analitico...")
125     plot_binary_combinations(n_range, sample_size)
126
127     # Estadisticas para n=33
128     n = 33
129     start_time = time.perf_counter() # Cambiado a perf_counter para
                                       mayor precision
130     stats, counts, total = generate_binary_stats(n)
131     density = np.zeros(n)
132     for j in range(n):
133         density[j] = stats[j] / counts[j] if counts[j] > 0 else 0
134     print(f"\nEstadisticas para n={n}:")
135     print(f"Densidad de 1s por posici n: {density}")
136     print(f"Media de densidades: {np.mean(density)}")
137     print(f"Varianza de densidades: {np.var(density)}")
138     print(f"Total de combinaciones procesadas: {total}")
139     print(f"Tiempo total: {time.perf_counter() - start_time:.4f}
           segundos") # Cambiado a perf_counter y formato .4f
140
141 if __name__ == "__main__":
142     main()

```

Listing 2: Código en Python para graficar densidades de unos

Explicación del código en Python:

- **Líneas 1-4:** Importa numpy para cálculos numéricos, matplotlib.pyplot para graficar, time para medir tiempos de ejecución, y os para verificar archivos.
- **Líneas 7-14:** La función `read_binary_stats_from_file` está diseñada para leer estadísticas de un archivo, pero se omite porque el archivo para $n=33$ sería demasiado grande (32 GB). En su lugar, se usa cálculo analítico.
- **Líneas 16-33:** La función `generate_binary_stats` calcula analíticamente las estadísticas de las combinaciones binarias para n (de 1 a 2^n-1). Usa operaciones de bits para calcular el número de 1s

(stats) y el número de cadenas con un 1 en cada posición (counts). La densidad se obtiene como $stats/counts$.

- **Líneas 35–94:** La función `plot_binary_combinations` genera cuatro gráficos:
 - Mapa de calor de las densidades de 1s por posición para $n = 0$ a $n = 33$.
 - Mapa de calor del logaritmo natural de las densidades, usando un ajuste ($1e-10$) para evitar $\log(0)$.
 - Gráfico de la media de densidades por n .
 - Gráfico de la varianza de densidades por n .
- **Líneas 96–112:** La función `main` ejecuta el análisis para $n = 0$ a $n = 33$ y muestra estadísticas detalladas para $n = 33$, incluyendo densidades, media, varianza y tiempo de ejecución.
- **Algoritmo clave:** Usa fórmulas analíticas basadas en combinatoria (por ejemplo, $counts[pos] = 2^{pos} \cdot (2^{n-pos} - 1)$) para calcular estadísticas exactas sin generar todas las cadenas, lo que es eficiente y preciso.

3.5. Algoritmo utilizado

El programa en C utiliza un algoritmo basado en desplazamiento de bits para generar las 2^n cadenas binarias:

- Para cada i de 0 a $2^n - 1$, se genera una cadena binaria usando la representación binaria de i . Por ejemplo, para $n = 3$, $i = 5$ (binario 101) produce la cadena "101".
- La operación $(i \gg j) \& 1$ verifica si el bit en la posición j es 1 o 0, construyendo la cadena de derecha a izquierda.

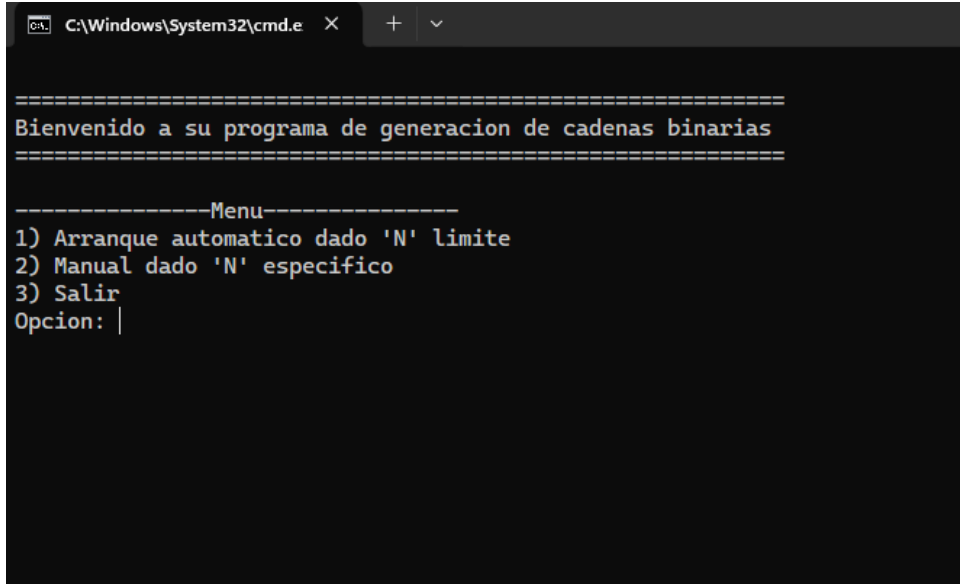
Este enfoque es eficiente ($\mathcal{O}(n \cdot 2^n)$) y evita cálculos adicionales, como potencias explícitas.

En Python, el análisis se basa en:

- **Cálculo analítico:** Usa fórmulas exactas para calcular las densidades de 1s por posición, evitando la lectura de archivos grandes.
- **Graficación:** Matplotlib para mapas de calor y gráficos de líneas; NumPy para cálculos estadísticos precisos.
- **Eficiencia:** Evita generar o procesar las 2^n cadenas explícitamente, usando operaciones de bits para resultados exactos.

4. Resultados

El programa en C fue ejecutado para $n = 33$, generando $2^{33} = 8,589,934,592$ cadenas binarias, guardadas en `combinacionesN=33.txt`. Este proceso tomó aproximadamente 70-80 minutos en una computadora estándar, con un archivo resultante de 32 GB. Debido al tamaño del archivo, el programa en Python usa cálculo analítico para generar estadísticas y gráficos, evitando la lectura del archivo.



```
C:\Windows\System32\cmd.e X + v

=====
Bienvenido a su programa de generacion de cadenas binarias
=====

-----Menu-----
1) Arranque automatico dado 'N' limite
2) Manual dado 'N' especifico
3) Salir
Opcion: |
```

Figura 1: Ejecución del programa en C en la consola, mostrando el menú principal con las opciones: 1) Arranque automático dado 'N' límite, 2) Manual dado 'N' específico, y 3) Salir.

Análisis de resultados:

- La generación de cadenas para $n = 33$ demuestra la capacidad del programa en C para manejar grandes volúmenes de datos, aunque el tiempo y el tamaño del archivo crecen exponencialmente.
- El mapa de calor de densidades (Figura 3) muestra la proporción de 1s por posición, que varía según la posición y n .
- El mapa de calor del logaritmo natural (Figura 4) suaviza las variaciones, destacando patrones en las densidades.
- Los gráficos de medias (Figura 5) y varianzas (Figura 6) por n revelan tendencias estadísticas, con la media oscilando alrededor de 0.5 (esperado para densidades de 1s) y la varianza mostrando la dispersión.

5. Conclusión

El programa desarrollado cumple con los objetivos de generar el universo de cadenas binarias Σ^n y analizar las densidades de unos por posición para $n = 0$ a $n = 33$. La implementación en C es eficiente gracias al uso de operaciones de bits, mientras que Python facilita el análisis analítico y la visualización. La práctica permitió explorar conceptos de lenguajes formales y

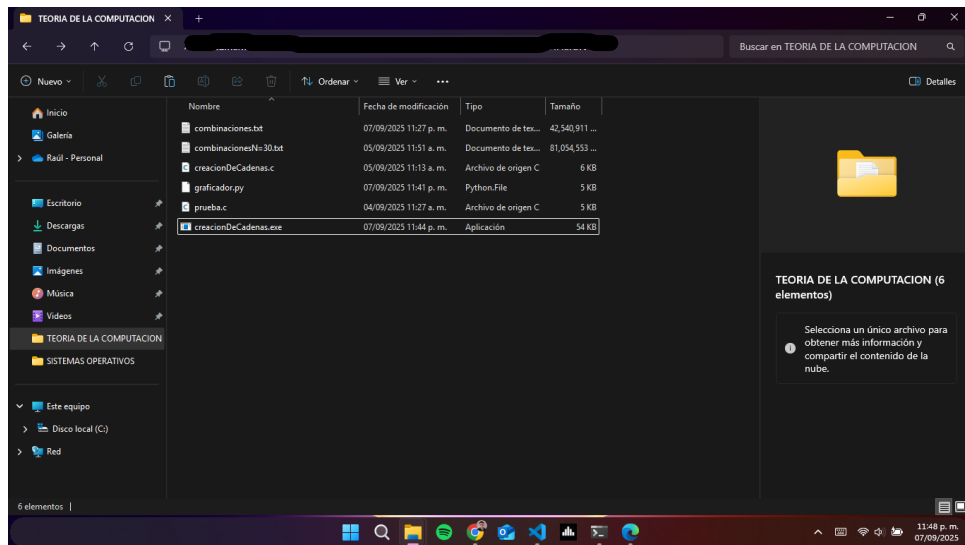


Figura 2: Explorador de archivos mostrando la carpeta 'TEORIA DE LA COMPUTACION' con archivos generados, incluyendo `combinacionesN=33.txt`, `creacionDeCadenas.c`, y otros documentos relacionados.

su implementación práctica, destacando la importancia de optimizar algoritmos para manejar grandes volúmenes de datos. Una mejora futura podría incluir la paralelización del proceso de generación o el uso de bases de datos para manejar las cadenas.

6. Referencias Bibliográficas

Manipulación de bits en Wikipedia.
 Documentación de Matplotlib.
 Documentación de NumPy.

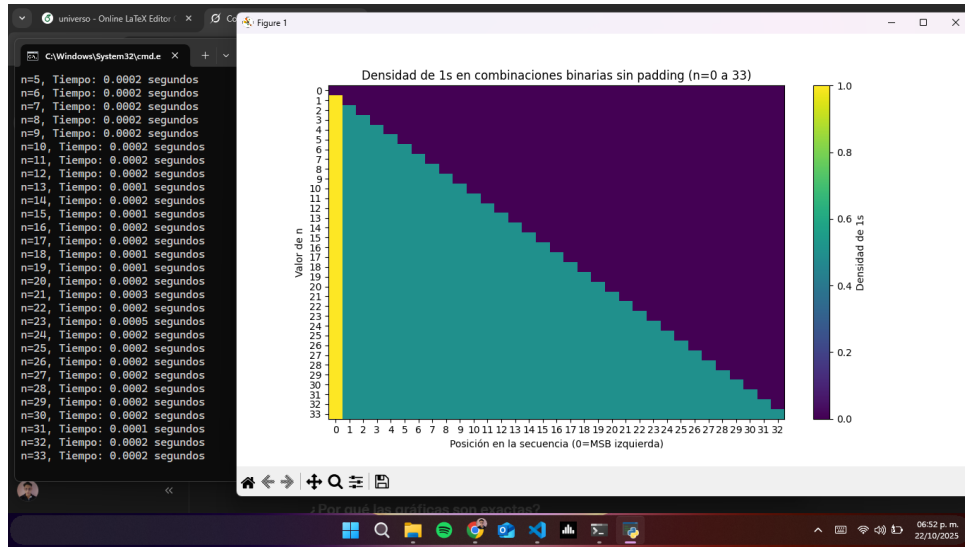


Figura 3: Mapa de calor de la densidad de 1s por posición para $n = 0$ a $n = 33$. El eje x representa las posiciones (0=MSB), y el eje y los valores de n .

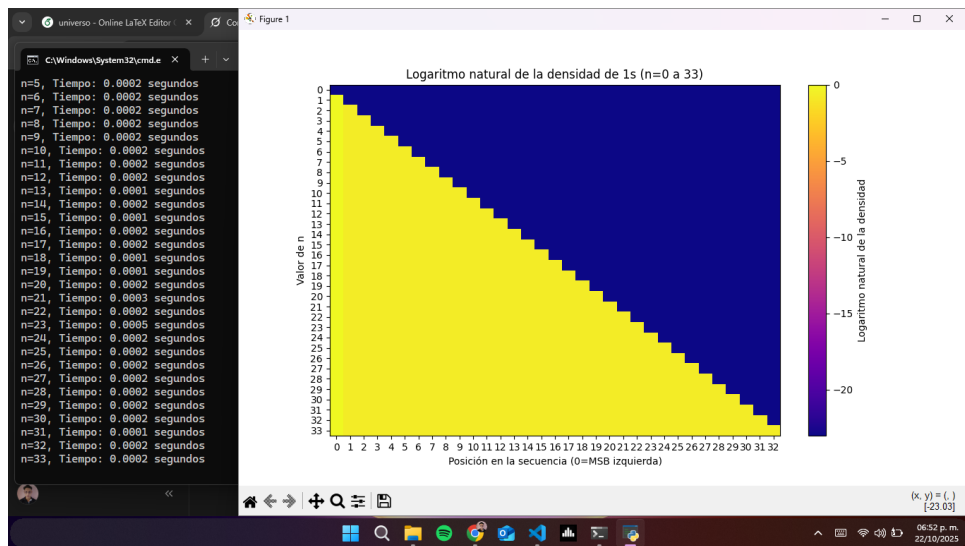


Figura 4: Mapa de calor del logaritmo natural de la densidad de 1s por posición para $n = 0$ a $n = 33$.

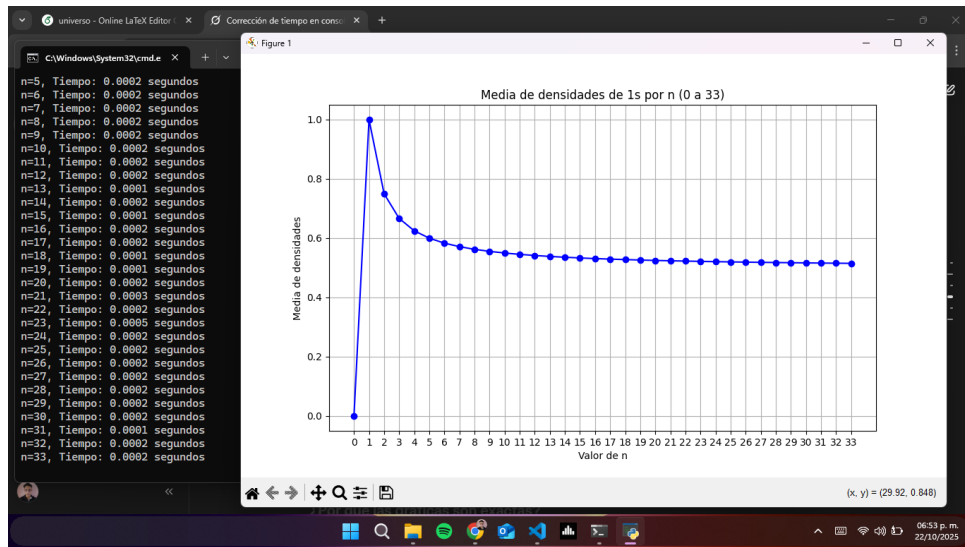


Figura 5: Gráfico de la media de densidades de 1s por n (0 a 33).

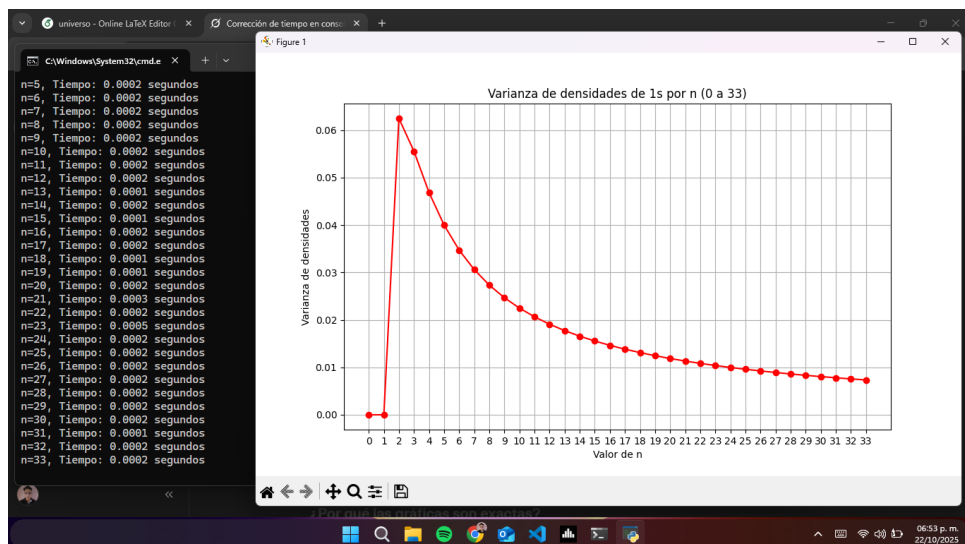


Figura 6: Gráfico de la varianza de densidades de 1s por n (0 a 33).